National Real-Time Payments

When you are building a platform that aims to replace cash and handle 150 million registered users alongside peak volumes of 50,000+ transactions per second (TPS), We aren't just building an app, we are building critical national infrastructure.

1. Systems and subsystems

To support the extensive functional requirements, we need distributed, microservices-based architecture to ensure independent scaling and fault tolerance.

- **User & Identity Management:** Handles user onboarding, customer verification (KYC), and securely linking multiple bank accounts per user.
- **Core Transaction Engine:** The high-speed heart of the system that processes real-time peer-to-peer (P2P) payments and QR-based peer-to-merchant (P2M) payments.
- **Bank Integration Gateway:** A dedicated abstraction layer that manages integration with all major banks to route debit and credit instructions.
- **Settlement & Reconciliation System:** An asynchronous ledger system that manages inter-bank settlements, scheduled settlements, and chargebacks.
- **Fraud & Risk Engine:** Conducts real-time fraud detection and risk scoring without adding latency to the transaction flow.
- **Merchant Management System:** Handles merchant registration, verification, QR code generation, and daily settlement reporting.

2. Information Exchanged

- **High Speed, Low Volume (Per Payload):** Transaction initiation requests (amount, sender, receiver) and transaction status updates (success, pending, failed) require rapid exchange. Because end-to-end latency must be under 2 seconds, these payloads must be lightweight and transmitted via high-speed protocols (like gRPC or WebSockets).
- **High Volume, High Consistency:** Financial debits and credits sent to banking systems. These require absolute transactional integrity to ensure zero data loss.

- **Eventual, High Volume:** Audit logs, daily settlement reports, and regulatory reporting can be batched and exchanged asynchronously, as they do not block the user flow.

3, 4 & 5. Failures, Mitigations, and Contingencies

Because the system requires 99.999% availability, we must assume things will break and design for survival.

- **External Dependency Failures (Bank Outages):** Banks will inevitably face downtime. The system is explicitly required to work during partial bank outages.
- ***Mitigation:*** We will use **circuit breakers** to stop bombarding a failing bank with requests, preventing cascading network failures.
- ***Contingency:*** We can implement **deferred settlements**, processing the user's transaction on our ledger and settling with the bank once they are back online, supported by proactive **user communication strategies** ("Your bank is delayed, but we secured your payment").
- **Integration & Technical Failures:** Server crashes or database failures.
- ***Mitigation:*** The architecture must feature **no single point of failure**. We will rely on **redundancy** across multiple data centers and **automatic failovers**.
- ***Contingency:*** If automated systems fail, robust **manual reconciliation** processes and dispute management systems must be activated to resolve discrepancies.

6. System Behavior: Normal vs. Peak Load

- **Normal Daily Load:** The system efficiently processes baseline traffic for the 40 million daily active users, relying on standard auto-scaling rules to maintain the <2 second latency requirement.
- **National Peak Events:** During festivals, salary days, or tax deadlines, traffic can spike up to 10x. To survive this, the system relies on aggressive **horizontal scalability**. Furthermore, the system will employ **graceful degradation**; for example, it might temporarily disable non-critical features (like fetching older transaction history or generating heavy admin dashboards) to dedicate 100% of computing power to core payment routing.

7. Architectural Trade-offs

- **Consistency vs. Availability:** For the user interface, we lean toward Availability (showing a transaction is "Pending" quickly). For the core financial ledger, we choose strict Consistency to guarantee zero data loss, even if it adds slight latency.
- **Real-time vs. Eventual Processing:** Core payments are strictly real-time. However, to meet the constraint of not delaying transactions, fraud detection must be carefully managed. We trade strict real-time fraud blocking for an asynchronous approach: we block known, high-risk flags instantly, but process complex risk scoring eventually, raising alerts for manual review post-transaction.
- **Cost vs. Reliability:** Achieving 99.999% availability and zero single points of failure is incredibly expensive in terms of infrastructure. We trade higher operational costs for the guarantee of national-scale reliability.
- **Security Depth vs. User Experience:** Strict PCI DSS compliance, ISO 27001, and strong authentication can create user friction. We trade off traditional multi-step passwords for advanced biometric integrations (FaceID/Fingerprint) to maintain a fast UX while keeping deep security.

8. Global Implementations & USPs

When we look at global success stories like India's UPI or Brazil's Pix, their rapid adoption was driven by extreme interoperability and mobile-first simplicity.

Recommended USPs (Unique Selling Propositions) for this platform:

- **"Future-Proof" Cross-Border Abstraction:** The requirements state that the system will eventually be replicated for international payments, requiring currency conversion, foreign banking integration, and international regulatory compliance.
- **Impact/ROI***:* By designing the core messaging protocol around global standards (like ISO 20022) from day one, we avoid millions of dollars in technical debt and years of refactoring later, drastically accelerating the time-to-market for international expansion.
- **Offline Payment Capabilities:** Enabling secure, low-value proximity payments via Bluetooth or NFC when internet connectivity drops.

- **Impact/ROI:** Highly valuable in rural or high-density areas with poor network coverage, directly driving up Daily Active Users and transaction volume.